

**HW3: Acceleration of finite impulse response (FIR) filtering by blockwise processing**

Due **Sunday** Oct. 24, 2021

Prof. Yi-Wen Liu

In this homework, you will gain hands-on experience in implementing low-pass and band-pass filtering. You should compare two methods: (a) direct convolution in the time domain, and (b) accelerating it by FFT.

Resource:

<boxing.wav> is an audio file that contains singing with background music

**Description:**

Suppose that  $x[n]$  denotes the input signal, the starter code uses `fir1()` to design a finite impulse response (FIR) low-pass filter of Type I. Let us denote the impulse response of this filter as  $h[n]$  and please use `conv()` or `filter()` to perform the convolution. Let us also denote the output as

$$y[n] = x[n] * h[n] = \sum_{k=0}^L x[n-k]h[k]. \quad (\text{Eq. 1})$$

1. (10%) Suppose that the length of the filter is  $L + 1 = 129$  samples and the audio is 10 second long with 44.1 kHz sampling rate. Use `tic` and `toc` to measure how long it takes for your computer to finish filtering using the MATLAB function `conv()` or `filter()`.

2. (20%) Without using the functions, implement convolution with `for` loop(s) using the direct definition in (Eq. 1). How long does it take? Is it slower than MATLAB's functions? Make sure that the output signal  $y[n]$ , nevertheless, is the same as in Problem 1.

3. (60%) Use the rectangular window to capture non-overlapping frames as follows,

$$x^{(k)}[n] = x[n]w[n - kM], \quad (\text{Eq. 2})$$

where  $k$  is the frame index,  $M$  is the frame length, and  $w[n]$  denotes a rectangular window  $w[n] = 1$  for  $1 \leq n \leq M$  and  $w[n] = 0$  otherwise. Note that the following identity holds:

$$x[n] = \sum_{k=0}^{\infty} x^{(k)}[n] \quad (\text{Eq. 3})$$

since all the blocks do not overlap. Then, convolving both sides of Eq. 3 with  $h[n]$  and applying the linearity of convolution, we have

$$y[n] = \sum_{k=0}^{\infty} h[n] * x^{(k)}[n]$$

Denote and define the  $k^{\text{th}}$  output frames as  $y^{(k)}[n] = (h * x^{(k)})[n]$ . The following steps describe how to accelerate convolution by processing one frame at a time and operating in the frequency domain.

- (a) Try  $M = 256$ . Append zeros to both  $h[n]$  and  $x^{(k)}[n - kM]$  in the end to create two arrays `h_zp` and `x_win_zp` of length 512, respectively.
- (b) Multiply the FFT of `h_zp` and `x_win_zp` in the frequency domain.
- (c) Take the inverse transform of the result of (b) to get a block of the output of length 512 in the time domain. Actually, the first  $M + L - 1 = 383$  samples are all we need for further processing. Please verify that the 384<sup>th</sup> sample and beyond are all zeros.
- (d) Overlap and add the output appropriately to obtain your output signal. When the shift and add are implemented correctly, your result should be identical to direct convolution in Problem 1. Please verify it.
- (e) Use `tic` and `toc` to check whether going through the frequency domain indeed accelerates convolution. In class, we've shown that theoretical analysis suggests so.
- (f) Investigate how the computation efficiency depends on the choice of windowing length  $M$ .

4. (10%) Change the window type from rectangular to Hann and repeat Problem 3. What changes do you need to make to assure that the output still equals to direct convolution in the time domain?

For this homework, please turn in your code and a brief report.